

分散 N-Queens ベンチマーク

Mac AIPL ランタイムと Raspberry Pi Xinu クラスタの協調実行

児玉靖司

法政大学経営学部 yass@hosei.ac.jp

2026 年 6 月 30 日

Abstract

型付き AI エージェント言語 AIPL のインタプリタ実行系 (Mac 上の Py-I ランタイム) と、ベアメタル Xinu を載せた Raspberry Pi 実機 3 台 (rpi3/rpi4/rpi5) から成る WiFi クラスタを対象に、古典的探索問題である N -Queens を題材とした分散ベンチマークを行った。第一クイーンの列位置で探索木を互いに素な部分に分割し、各ワーカーへ列範囲を割り当てて並行実行・部分解の合算を行う方式を実装した。4 構成 (Mac 単体 / +rpi3 / +rpi3+rpi4 / +rpi3+rpi4+rpi5) について N を漸増させて計測した結果、インタプリタ実行系から native Xinu クラスタへ計算をオフロードすることで最大 $\times 637$ ($N = 11$, rpi3 構成) の高速化を確認した。全構成の総解数は OEIS A000170 の既知値と完全に一致し、分散実行の正当性を検証した。

1 背景と目的

AIPL は型付きの AI エージェント指向言語であり、その参照実装 (Py-I ランタイム) は動的型検査とアクタ・メッセージングを優先したインタプリタ実行系である。表現力と検証容易性の代償として実行速度は低く、重い数値探索には不向きである。一方、筆者らは Raspberry Pi 3/4/5 上で動作するベアメタル Xinu に SMP ワーカープール (コア 0=OS, コア 1-3= 計算ワーカー) と HTTP アクタ実行系を実装してきた。本実験の目的は、遅いインタプリタ実行系の計算を、ネットワーク越しに native な Xinu クラスタへ分散オフロードするという構成の有効性を、再現性のある定量ベンチマークで示すことである。

題材は N -Queens ($N \times N$ 盤に互いに利かない N 個のクイーンを置く配置数) とした。解の総数は第一クイーンを置く列 $k \in \{0, \dots, N-1\}$ ごとの部分解数の互いに素な和であり、列ごとに独立に数え上げできるため分散に適する。

2 実験環境

- **Mac**(オーケストレータ兼 AIPL ワーカー): AIPL の Py-I ランタイムが `nqueens_subset.abcl` (ビットマスク無しの素朴な再帰探索) を解釈実行する。
- **rpi3** (BCM2837 / Cortex-A53): 単一コアで列範囲を直接計算 (WiFi, :8080)。
- **rpi4** (BCM2711 / Cortex-A72 $\times 4$): SMP ワーカープールで列を 4 コアに分散。
- **rpi5** (BCM2712 / Cortex-A76 $\times 4$): 同上, 最新・最速コア。

各 Xinu ボードは native ルート GET /nqpart?n=N&c0=A&c1=B を備え、第一クイーンを列 $[A, B)$ に置く配置数を数えて `solutions=.. ms=.. cores=..` を返す。rpi4/rpi5 ではこの列範囲を `smp_parallel_sum` で各コアに分配する。ボード側 native 実装はビットマスク・バックトラッキングであり、cc-JIT の 250ms 暴走デッドラインを回避して正確な計数を保証する。

3 手法

1. **分割**： N 個の第一クイーン列 $\{0, \dots, N-1\}$ をアクティブなワーカー数でほぼ等分し，連続した列ブロックを各ワーカーに割り当てる（rpi4/rpi5 は受け取ったブロックを内部でさらに各コアへ SMP 分散する）。
2. **Mac の扱い**：AIPL インタプリタは 1 ボードの $\sim 10^4$ 倍遅いため，複数ボード構成では Mac は列を担当せずオーケストレータに徹する（後述の通り測定上の必然）．Mac 単体構成でのみ Mac が全列を解釈実行する。
3. **ウォームアップ**：計測前に各ボードへ極小要求 ($N=4$) を 1 回送り，単一スレッド webactor の初回往復遅延（最大 ~ 35 s）を計測から除く。
4. **計測**：オーケストレータが並行ディスパッチ開始から最後のワーカー応答までの実時間（wall-clock）を測る．これが各構成の所要時間である。
5. **検証**：各部分解の総和を OEIS A000170 の既知値と照合し，全 $(, N)$ で一致を確認した。

4 結果

4.1 所要時間（wall-clock 秒）

N	Mac (AIPL)	+rpi3	+rpi3+rpi4	+rpi3+rpi4+rpi5
8	0.579	0.018	0.119	0.110
9	1.67	0.020	0.136	0.107
10	7.30	0.027	0.138	0.112
11	40.46	0.064	0.153	0.161
12	–	0.256	0.234	0.160
13	–	2.34	0.718	1.51

全 $(, N)$ で総解数は既知値と一致した（正当性 100%）．Mac 単体は N の増加に対し急峻に悪化する（解釈実行のため探索木サイズにほぼ比例）一方，クラスタ構成は native 実行により桁違いに高速である。

4.2 Mac 単体に対する高速化倍率

N	+rpi3	+rpi3+rpi4	+rpi3+rpi4+rpi5
8	$\times 33$	$\times 5$	$\times 5$
9	$\times 81$	$\times 12$	$\times 16$
10	$\times 267$	$\times 53$	$\times 65$
11	$\times 637$	$\times 265$	$\times 251$

4.3 最大 N での列割り当てと端末計算時間

$N = 13$				
構成	ワーカー	列範囲	部分解	端末計算 [ms]
+rpi3	rpi3	$[0, 13)$	73712	0
+rpi3+rpi4	rpi3	$[0, 7)$	40891	0
	rpi4	$[7, 13)$	32821	480
+rpi3+rpi4+rpi5	rpi3	$[0, 5)$	25436	0
	rpi4	$[5, 9)$	30186	274
	rpi5	$[9, 13)$	18090	124

5 考察

(1) **インタプリタ・オフロードが支配的** 最大の効果は AIPL 解釈実行から native Xinu への計算移譲である。Mac 単体は $N=10$ で約 7.3s, $N=11$ で約 41s を要したが、同じ問題を 1 ボードに投げるだけで応答は 0.1s 未満に収まる。これは「遅い高水準実行系+速いエッジ実機クラスタ」という非対称構成の実用的価値を示す。

(2) **ネットワーク律速と計算律速の交差** ボード同士の比較では 2 つの領域が現れる。小さい N では計算が一瞬で終わり、所要時間は HTTP 往復遅延に支配される（最遅ボードの RTT が律速）。このためボードを増やすとかえって遅くなることがある（協調オーバーヘッド）。 N が大きく計算が支配的になると、列分割による真の並列性が効き、SMP コアを持つ rpi4/rpi5 を加えるほど makespan が縮む。

(3) **ヘテロジニアス・クラスタ** 等分割では単一コアの rpi3 が割当ブロックのボトルネックになりやすい。端末計算時間（表）から、同じ列数でも rpi3（1 コア）対 rpi4/rpi5（4 コア）で処理時間が大きく異なることが読み取れる。速度に比例した重み付き分割は今後の課題である。

6 結論

AIPL インタプリタ実行系と native Xinu クラスタを協調させ、 N -Queens を第一クイーン列で分割して分散実行する系を実装・計測した。全構成で OEIS A000170 と一致する正しい解を得つつ、インタプリタから実機クラスタへのオフロードにより大幅な高速化を達成した。本結果は、重い計算を高速なエッジ実機群へ委譲する AIPL 実行モデルの妥当性を支持する。

参考：N-Queens 解数列 OEIS A000170 <https://oeis.org/A000170>, 測定スクリプト `nq_cluster_bench.py`, 生データ `results.json`.